

EXTRACTION AUDIT REPORT

Glass-processing platform: PDF upload and Telegram Inbox extraction pipeline

RISK VERDICT	RISKY
Scope	Audit only. Extraction-related code only. No files modified.
Primary concern	Syntactically valid but wrong extraction can be persisted as a normal draft without a hard quality gate.

Prepared 10 July 2026

1. Executive Summary

Verdict: risky. Not safe enough for unattended factory production.

The primary PDF transport is modern and correct: PDF bytes are base64-encoded inside a Responses API `input_file`. Upload and Telegram PDFs share the same active extraction service, model, prompt, schema, normalization, and validation.

The dangerous part is after the model responds

- Validation mostly warns; it rarely blocks.
- Missing quantity becomes 1; invalid area becomes 0.0; missing dimensions are allowed.
- Implausible order numbers are accepted unless one specific KELI pattern is recognized.
- Piece and area total mismatches generate warnings but do not stop persistence.
- Suspicious results are saved immediately as drafts.
- Targeted OCR repairs are attached after canonical saved rows have already been finalized.
- No retry is triggered by low confidence, missing rows, total mismatch, or suspicious order number.

The platform is not broken in the sense that it cannot extract PDFs. It is operationally unsafe because valid-looking but wrong results can be persisted and presented for review without a hard extraction-quality gate.

2. Current Extraction Flow

Drag-and-drop upload

Stage	Active path
Frontend entry	<code>docs/js/app.js:17983</code> drop handler or file picker -> <code>handlePdfExtraction(file)</code>
HTTP request	<code>callPdfAPI()</code> posts multipart file to <code>POST /extract_pdf</code>
Backend route	<code>backend/app.py:2401</code> <code>extract_pdf()</code> reads bytes and calls <code>_extract_order_file_bytes(source='pdf')</code>
OpenAI	<code>backend/llm.py:220</code> <code>call_llm_for_pdf_base64_visual()</code>
Processing	Pydantic -> KELI metadata normalization -> dimension repair -> row normalization -> validation -> area checks -> targeted OCR
Persistence	<code>insert_extraction_with_rows()</code> creates or updates a draft before response
Return	<code>applyExtractionResult()</code> displays rows; later approval calls <code>/orders/{id}/approve</code>

Telegram Inbox

Stage	Active path
Entry	<code>POST /webhook/telegram</code> -> <code>_handle_telegram_update()</code>
Intake	Durable Telegram record, queueing, original file download and storage
Worker	<code>_process_telegram_queue_job_sync()</code> reads stored bytes
Shared service	Calls the same <code>_extract_order_file_bytes(source='telegram')</code> as upload
Persistence	Creates a draft, links Telegram file, performs duplicate heuristics
Outcome	Marks record extracted and tells sender to review in platform

3. How PDFs Are Sent to OpenAI

Classification: B - Acceptable base64 transport. The active path does not paste base64 into prompt text. It puts an inline PDF data URL into a proper Responses API file input.

```
pdf_b64 = base64.b64encode(pdf_bytes).decode("ascii")
pdf_data_url = f"data:application/pdf;base64,{pdf_b64}"

{
  "type": "input_file",
  "filename": filename or "upload.pdf",
  "file_data": pdf_data_url,
}
```

Evidence: backend/llm.py:224-247. The prompt also explicitly says the PDF is attached as `input_file` and should be read visually and structurally (backend/prompts.py:79-82).

Secondary modes exist: legacy OCR renders pages to PNG, transcribes them, then sends text to a second model; image intake uses `input_image`. PDF base64 stored in the database is persistence, not prompt dumping.

4. OpenAI API Usage

Property	Finding
Native PDF endpoint	POST https://api.openai.com/v1/responses
Model	EXTRACTION_MODEL, default gpt-5.4-nano
Structured output	Responses text.format json_schema, strict=true
Temperature / reasoning	No temperature or reasoning settings on native PDF call
Transport client	Direct httpx.post
Text / OCR parsing	Chat Completions via OpenAI SDK, temperature=0.0
OCR model	OCR_MODEL, default gpt-5.4-mini
Request retry	Read timeouts only; same model and same payload
Errors	Usually surfaced; legacy per-page failures become warnings and extraction continues

Assessment: native PDF usage is modern, but the module mixes Responses API, Chat Completions, raw HTTP, SDK calls, and separate parsers. The so-called preferred-model fallback contains only one model.

5. Prompt Audit

Strengths

- Strict JSON requested.
- All rows and pages requested.
- Order and type propagation described.
- Missing dimensions should be blank rather than guessed.
- Quantity, area, warnings, and top-level confidence are requested.

Weaknesses and risk

- No order-number candidate hierarchy and no general rule preferring a labeled R-YY-NNNN over a bare number such as 0075.
- The known 0075 issue is fixed deterministically only for a specific KELI wording and usable text layer.

- No red index, handwritten client number, or color/location semantics.
- No semantic test distinguishing positions from client or document numbers.
- No row-level evidence, page reference, bounding box, or field-level confidence.
- Quantity uncertainty is converted into a production value by the instruction to default to 1.
- Area field does not distinguish printed, computed, per-piece, or total-for-quantity area.
- Multiple orders are flattened under one top-level order number and client name.
- No notes field and no backend acceptance policy for warnings or confidence.

6. Schema and Parser Audit

Area	Current behavior	Risk
Top-level	order_number, client_name, rows, warnings, confidence required	No multi-order grouping; confidence has no range
Row	order_number, type, dimension, position, quantity, area required	No notes, evidence, source value, page, or field confidence
Dimension	String in OpenAI schema; Pydantic permits empty	Missing dimension is structurally valid
Quantity	Integer; Pydantic minimum 1	Uncertainty becomes 1
Pydantic	ExtractionResult omits client_name and confidence	Schema definitions can drift; extras are ignored
Native parser	Strip fences then json.loads; invalid JSON fails	Good syntactic behavior, no local semantic schema validation
Legacy parser	Per-page parse errors become warnings	Rows from failed pages disappear
Corrections	Signature match can replace entire row list	Stale learned correction can overwrite good extraction

7. Validation Audit

Classification: weak for production safety; partial for review assistance.

What is checked

- Shape and numeric typing
- Dimension syntax
- Missing-field warnings
- Narrow high-quantity warning
- Possible split rows
- Type propagation
- Area/dimension mismatch
- Truncated-height suspicion
- Declared total mismatch when totals are found
- Telegram duplicate heuristics

What is not checked

- Client required
- General order-number plausibility

- Bare-number rejection
- Conflicting order candidates
- Realistic width/height and quantity ranges
- Duplicate positions
- Expected table row count
- Visual-table reconciliation
- Multi-client separation
- Confidence threshold
- Page coverage
- Evidence coverage
- Blocking total reconciliation

Invalid dimensions may be cleared to blank; invalid quantity becomes 1; invalid area becomes 0.0. The database insert requires only a nonempty row list.

8. Fallback / Retry Audit

- Native PDF input is first.
- Read timeouts retry the same request.
- Legacy OCR automatically runs only when enabled, PDF has no local text, and native extraction throws.
- `force_ocr=true` bypasses native extraction.
- Targeted OCR runs for missing critical fields after successful native extraction.
- Telegram retries certain failed jobs, normally repeating the same strategy.

Critical sequencing defect:

```
final_rows = apply_area_dimension_validation(normalized_rows)
targeted_ocr = _run_targeted_ocr_repair_for_missing_fields(...)
localized_response_rows = targeted_ocr["rows"]

insert_extraction_with_rows(
    rows=final_rows,    # OCR-repaired overlay rows are not canonical
)
```

Targeted OCR may produce fields such as `dimension_repaired` or `quantity_repaired`, but persistence still uses `final_rows`. The UI and stored metadata can therefore suggest a repair while the canonical saved row remains incomplete.

Safe future fallback

Native PDF -> strict evidence-bearing schema -> deterministic validation -> stronger model or rendered-page retry when suspicious -> candidate comparison -> accept only if validation passes -> otherwise explicit manual review.

9. Telegram vs Upload Differences

Concern	Upload	Telegram
Entry	<code>/extract_pdf</code>	<code>/webhook/telegram</code> and queue
Core extractor	Shared	Shared
File storage	Base64 source in extraction	Original file on disk plus extraction source
Retry	Request timeout retry	Queue retries plus request retry

Concern	Upload	Telegram
Duplicate checks	Hash/version behavior	Exact file and recent-order heuristics
Forced OCR	Available	Not in normal Telegram flow
Visibility	Warnings shown immediately	Bot says extraction finished; review is asynchronous
Size limit	No explicit extraction limit found	5 MB

Danger level: medium to high. Core consistency is good, but any safety fix must be implemented in the shared backend, not only the upload UI.

10. Known Bug Root Cause Hypotheses

Issue	Likely cause	Confidence
Missing / empty dimensions	Prompt and schema permit blank; validator warns; targeted OCR is not persisted canonically	High
Wrong order number / 0075	No general candidate hierarchy; KELI-specific normalization only	High
Quantity mismatch	Prompt and validator default uncertain or invalid quantity to 1	High
Area mismatch	Printed and computed semantics are mixed; mismatch only flags	High
Rows disappear	Model omission, legacy page failure continuation, exact-row deduplication	High
Multi-page loss	No page coverage evidence; legacy failed pages are skipped	High
Multi-order mixing	Flat rows plus one top-level client/order	High
Red number as position	No visual index or color semantics	Medium-high
OCR fights native	Parallel repaired fields instead of one reconciled canonical row	High
Repair not saved	Persistence uses final_rows from before targeted OCR	Very high

11. Dead Code / Duplicate Logic

Component	Classification	Recommended disposition
Native PDF Responses extractor	Active	Keep
Image Responses extractor	Active	Keep if image intake remains
Pasted-text extractor	Active	Keep separate from PDF flow
Legacy PDF OCR	Active behind flag / forced route	Dangerous; retain only until replacement is proven
Multi-page text parser	Active via legacy OCR	Fix partial-success behavior
/extract-pdf alias	Callable; unused by frontend	Document and deprecate later
Legacy Telegram handler	No caller found	Delete later after deployment verification
Model fallback loops	Only one model configured	Simplify or implement real escalation
llm.py and schema.py schemas	Both active	Consolidate one source of truth
Targeted OCR overlays	Active	Keep diagnostics, redesign canonical repair
Historical full-row correction	Active	Constrain; currently dangerous

12. File-by-File Map

File	Purpose / active functions	Risk and later action
docs/js/app.js	Drop/upload, /extract_pdf, display, approval	Add blocked/manual-review presentation
backend/app.py	Routes, shared orchestration, Telegram, validation sequence	Core acceptance and sequencing fix
backend/llm.py	OpenAI calls, schema, parsing, OCR fallback	Mixed APIs, page partial success, full-row correction
backend/prompts.py	Text and PDF extraction prompts	Add number hierarchy, evidence, red-index and notes rules
backend/schema.py	Pydantic extraction model	Consolidate schema; add semantic constraints
backend/validators.py	Row normalization and warnings	Stop silently defaulting dangerous values
backend/extraction_normalizer.py	KELI metadata and type cleanup	Broaden safely; preserve evidence
backend/dimension_repair.py	Truncated-height suggestion	Make accepted repairs canonical only after validation
backend/area_dimension_validator.py	Area/dimension comparison	Clarify per-piece versus total area
backend/utlis_text.py	Text normalization and total parsing	Heuristic totals; blanking invalid dimensions
backend/db.py	Draft and extraction persistence	Do not persist suspicious rows as ordinary clean drafts
backend/agents/skills/extraction_diagnostics.py	PDF location and OCR diagnostics	Estimated pages and overlay-only repairs
tests/test_smoke.py	Route and fallback coverage	Add semantic gate and persistence sequencing cases
tests/test_extraction_normalizer.py	Known KELI 0075 case	Add scanned and non-KELI variants
tests/test_validator_safety.py	Validator behavior	Add hard rejection / manual-review cases

13. Risk Ranking

Critical

- OCR repairs are not persisted into canonical rows.
- Missing/invalid critical fields can be saved.
- Quantity uncertainty becomes 1.
- Totals do not block persistence.
- No general protection against 0075-style order-number mistakes.
- Legacy pages can fail while partial extraction is accepted.
- Historical correction may replace the full row set.

High

- No validation-triggered escalation.
- No multi-order/client grouping.
- No duplicate-position detection.
- Dimension repair is advisory only.

- Invalid area becomes 0.0.
- Telegram can report completion for warning-laden extraction.
- Missing rows cannot be detected without declared totals.

Medium

- Mixed OpenAI APIs and clients.
- Duplicated schema definitions.
- Legacy routes and handler remain.
- Confidence has no acceptance threshold.
- Fallback output shapes differ.
- Large base64 source is stored in database fields.

Low

- base64_pdf_visual is a misleading method name.
- Fallback comments imply multiple models when only one exists.
- Duplicated prompt rules can drift.

14. Recommended Cleanup Plan

1. Introduce a deterministic acceptance decision without changing the OpenAI call.
2. Separate valid, suspicious, and invalid outcomes.
3. Prevent suspicious output from becoming an ordinary review-ready draft.
4. Make repaired values canonical only after deterministic revalidation.
5. Add order-number candidate validation and supplier/client reference separation.
6. Consolidate JSON Schema and Pydantic into one source of truth.
7. Add row evidence, page location, field confidence, notes, and area provenance.
8. Add duplicate-position, realistic range, page coverage, and multi-order checks.
9. Trigger stronger-model or rendered-page retry only when suspicious.
10. Compare native and fallback results instead of blind overwrite or merge.
11. Require manual review when retries disagree or totals remain mismatched.
12. After tests, deprecate legacy OCR, alias route, old Telegram handler, and misleading fallback code.

15. First Safe Fix Recommendation

Add one centralized, deterministic acceptance gate before `insert_extraction_with_rows()`.

Do not begin with a prompt rewrite. A better prompt can still produce errors and cannot guarantee production safety.

The gate should mark extraction suspicious when

- No plausible order number
- Missing dimension, type, or position
- Missing or defaulted quantity
- Nonpositive area

- Duplicate position
- Unrealistic dimensions or quantity
- Declared total mismatch
- Low confidence
- Unresolved OCR repair
- Conflicting order numbers or clients

For the first incremental change, suspicious extraction should remain available for review but must be explicitly marked `needs_manual_review`. It must not appear equivalent to a clean extraction, and the gate must not invent replacements.

Likely files

- `backend/app.py`
- `backend/validators.py`
- `backend/db.py`
- `backend/schema.py`
- `docs/js/app.js`
- `tests/test_validator_safety.py`
- `tests/test_smoke.py`

Why first: this reduces immediate factory risk without changing model behavior, OCR behavior, document handling, or the overall extraction architecture. It creates the safety boundary needed before prompts, retries, schemas, and legacy code are changed.